CprE 3810: Computer Organization and Assembly-Level Programming

Project Part 2 Report

Team Members: __Preston Gaskill_____

____N/A_____

_____

Project Teams Group #:_____N/A_____

Refer to the highlighted language in the project 1 instruction for the context of the following questions.

[1.a] Come up with a global list of the datapath values and control signals that are required during each pipeline stage.

IF/ID

1. s_Inst
2. currentInst

ID/EX

3. Current Instruction
4. Branch_Wire
5. Wire_ALUSrc
6. ALUControl
7. ImmType
8. ltSrc
9. MemWrite
10. PCSrc
11. RegDst
12. Halt
13. WritePCORResultSrc
14. ALUOutORPC
15. PCAddress
16. Regwrite
17. ZeroMuxWrite
18. Zero
19. Data 1
20. Data 2
21. FetchLogicAddress

EX/MEM

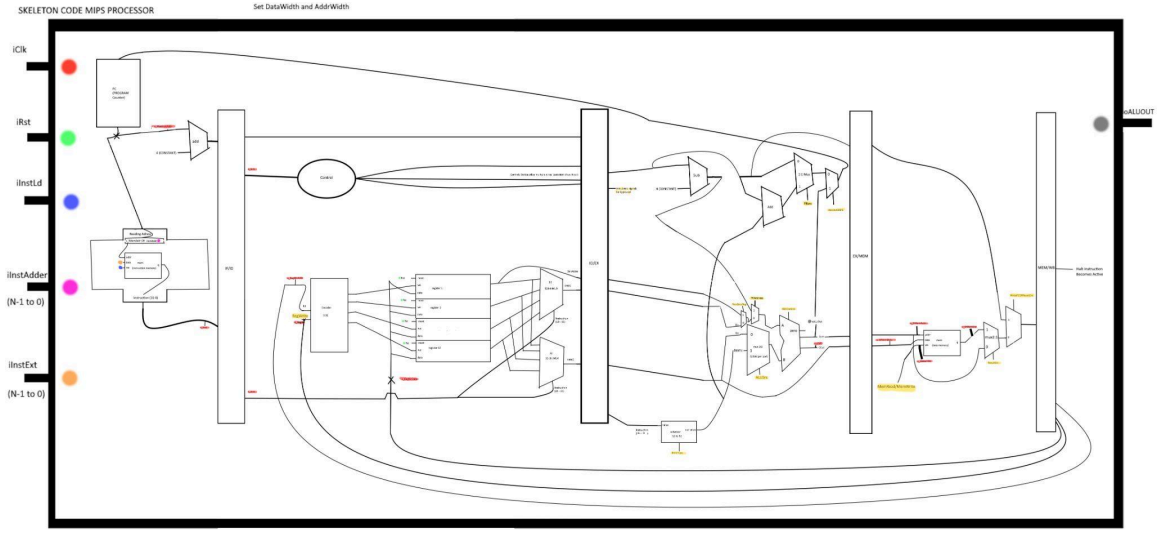
22. pcPlus4
23. ALUOut

24. Data 2
25. Opcode
26. Funct7
27. Rd
28. Regwrite
29. Memwrite
30. Resultsrc
31. writePCORResultSrc

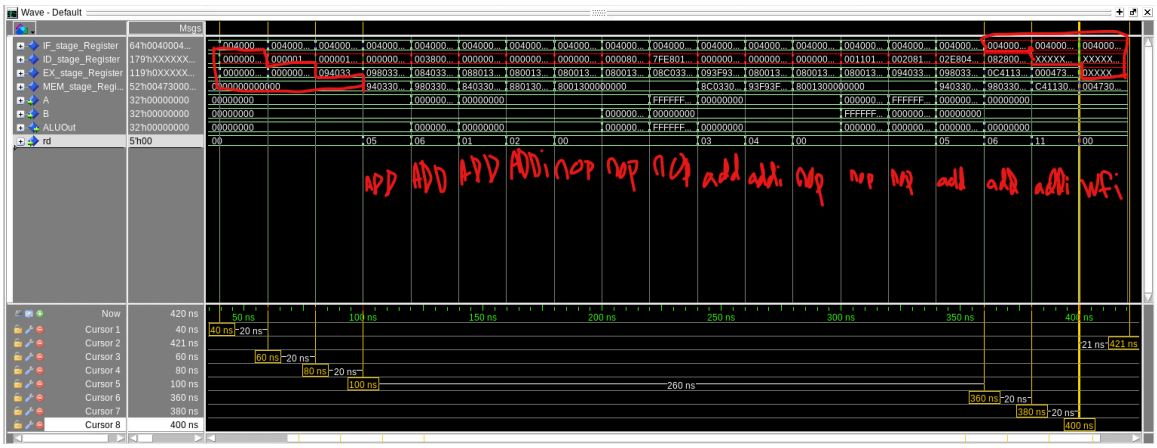
MEM/WB

32. Final data value for register
33. Opcode
34. Funct6
35. Rd location
36. Write enable signal

[1.b.ii] high-level schematic drawing of the interconnection between components.



[1.c.i] include an annotated waveform in your writeup and provide a short discussion of result correctness.



In the above diagram, you can see the ‘step’ looking output from the register stages and how they grow until the processor runs out of instructions (note that the control logic initially outputs a value at the start of the IF stage, but this doesn’t matter), and the ‘step’ pattern repeats in the opposite direction.

Our first write-back occurs after the 100ns cursor, and this continues until we reach our wfi instruction and the halt value is set (although it is not visible in the waveform, the change is instantaneous).

```
add x5, x0, x0
add x6, x0, x0
add x1, x0, x0

addi x2, x0, 7
nop
nop
nop
add x3, x2, x0

addi x4, x0, -3
nop
nop
nop
add x5, x4, x2

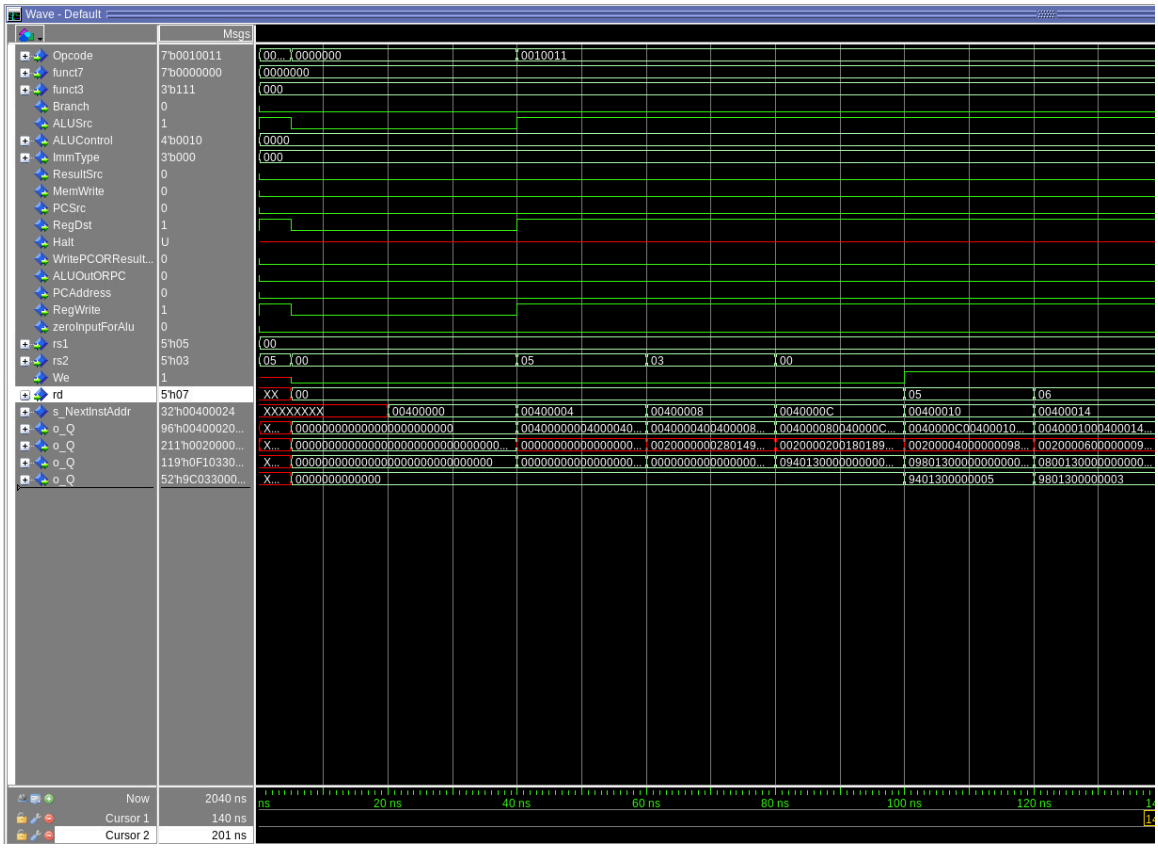
add x6, x2, x4

addi a7, x0, 93
wfi
```

[1.c.ii] Include an annotated waveform in your writeup of two iterations or recursions of these programs executing correctly and provide a short discussion of result correctness. In your waveform and annotation, provide 3 different examples (at least one data-flow and one control-flow) of where you did not have to use the maximum number of NOPs.

- **OMIT SOME**
- **Waveform for a program that contains all instructions supported by the SW pipeline**

The purpose of this program was to test every instruction that the CPU supports at least once.



Given the waveform above, you can see how, as the NextInstAddress increases, the opcode changes for each new instruction, and the corresponding rd, and ALU signals ALUOut, A, and B change. It will take approximately 5 cycles before we see instructions being loaded into our pipeline, but after that, we will be able to examine the aforementioned values to ensure that we are getting the correct output.

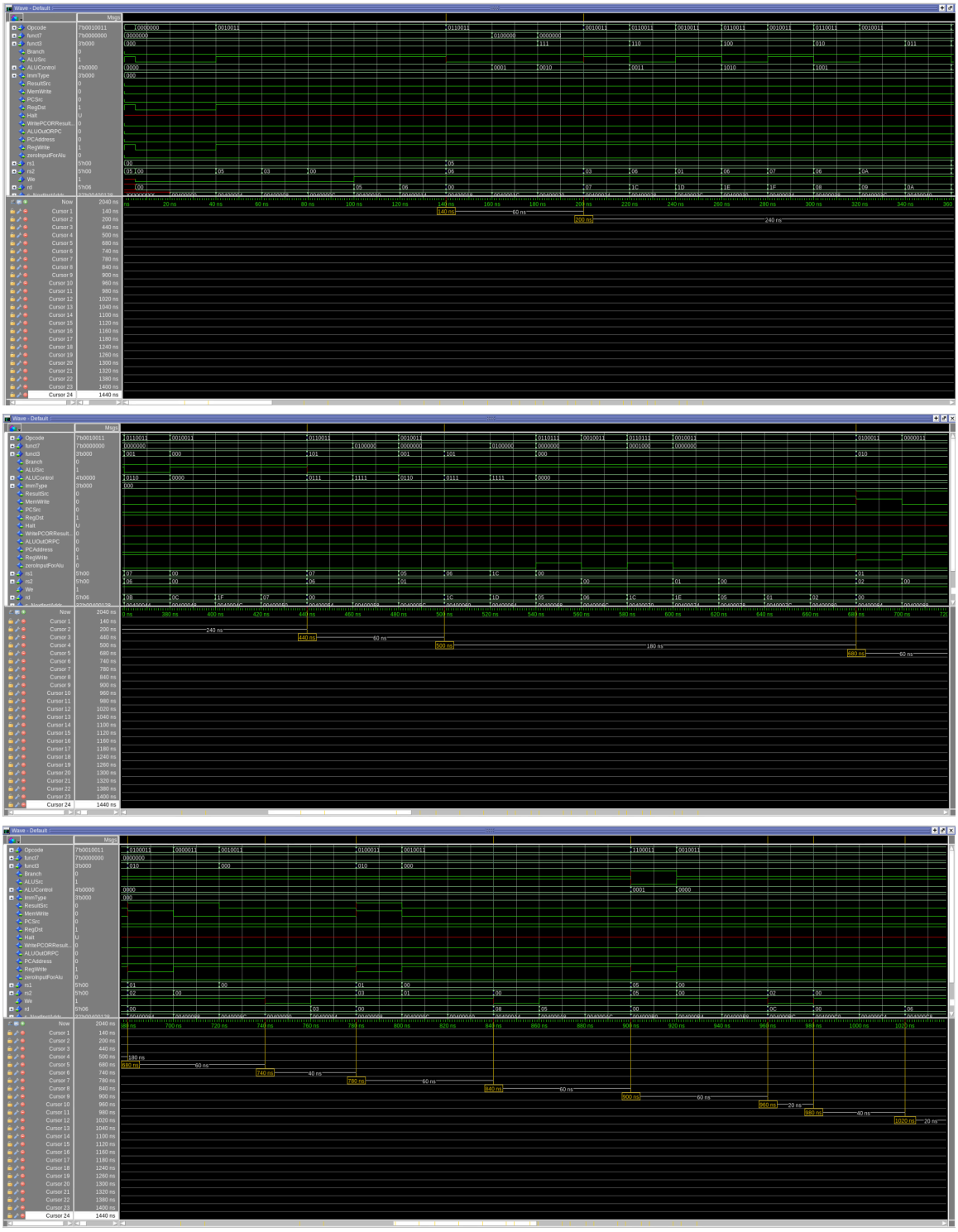
Example
Addi x5, x0, 5
Addi x6, x0, 3

The only sections where the instructions are not executed as they typically would be on a RISC-V single-cycle processor are during our no-ops, which help with our data dependencies. I marked those locations with cursors to make them easier to see. A cursor marks the start and finish of a series of no-op instructions.

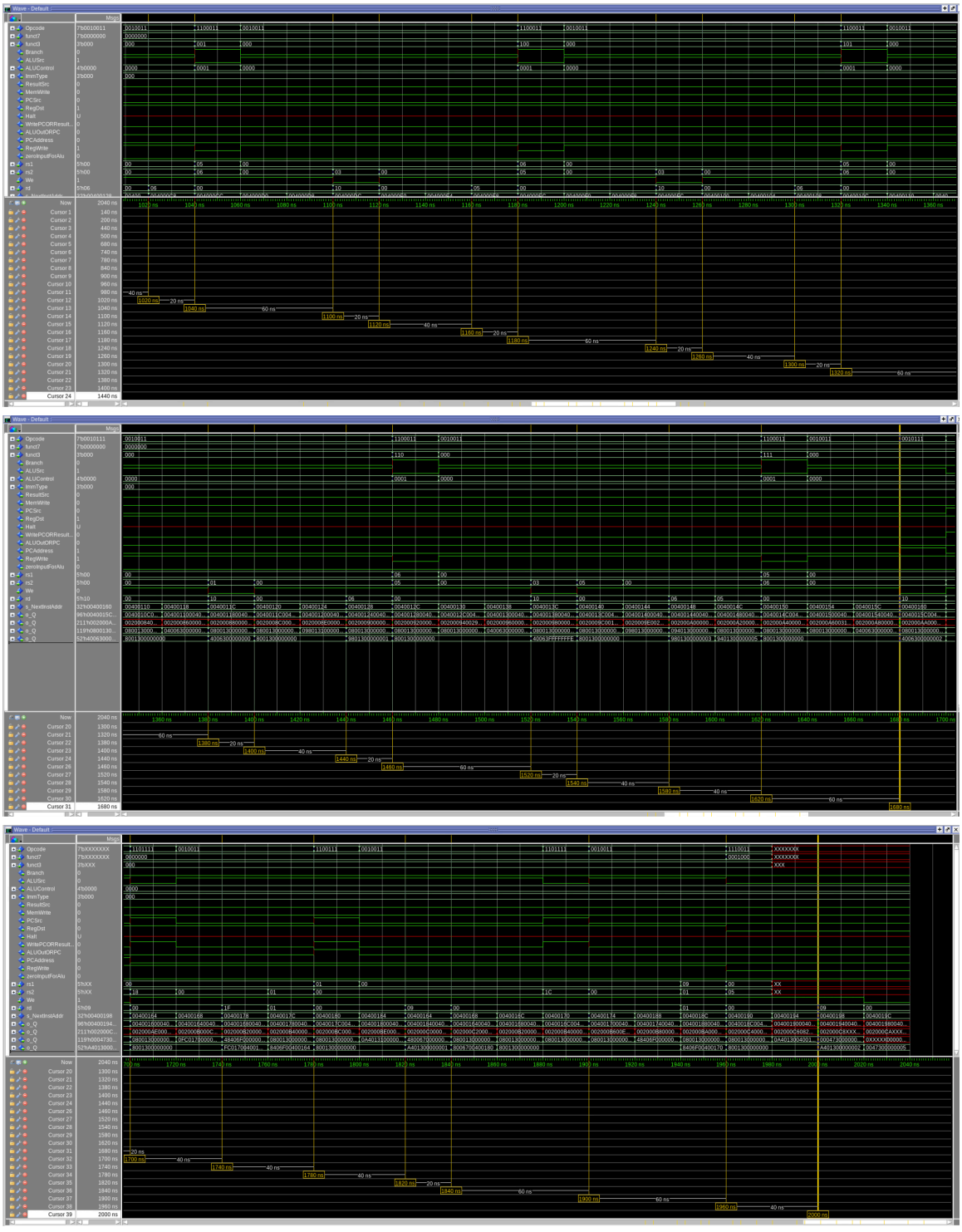
When these no-ops occur, we double-check that our NextInstAddress is still increasing normally, that our RD value is register 0, and that the MemWrite of the previous RD's cycle is set to 0.

For example, you can see how our NextInstAddress continues to increase from 0x00400018 onward to 0x00400020, and that our RD location is set to register 0, while our write signals remain at 0. This behavior is consistent across all of our waveform cursors until the WFI instruction is finally read and the halt instruction is set to high.

NOTE: All no-op series are either 3 or 2 no-ops total. If there is another RD = 00 after the cursor, it is probably a sw.

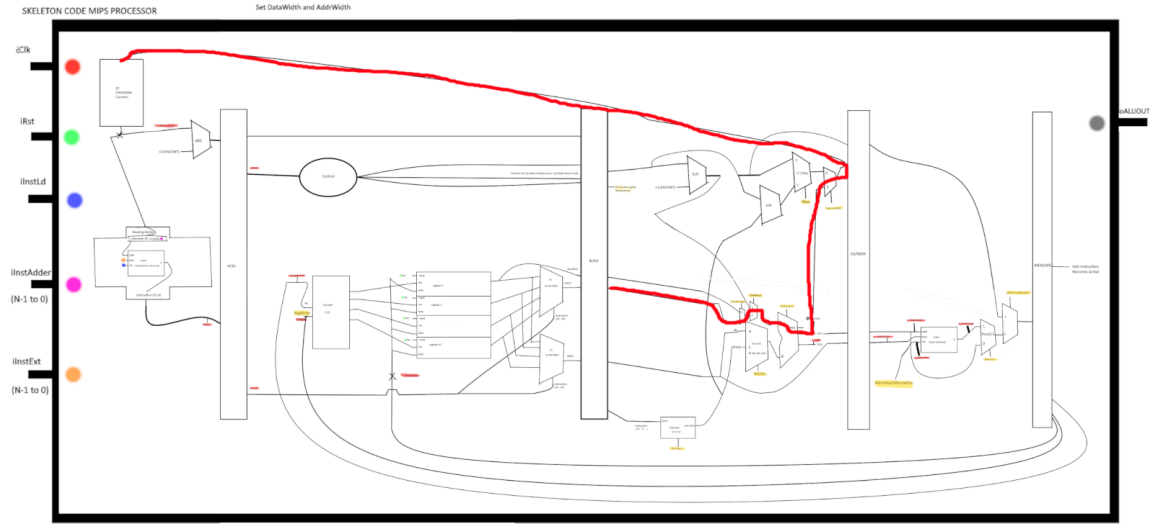


The first of our series of branches occurs here after 900 ns.

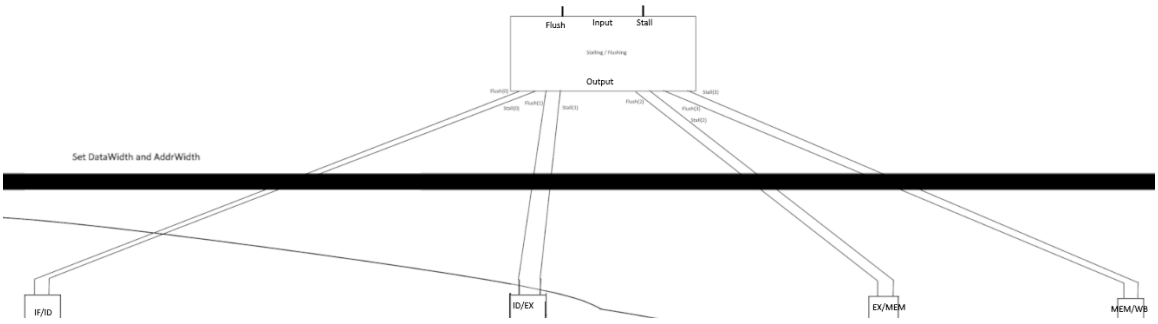


[1.d] report the maximum frequency your software-scheduled pipelined processor can run at and determine what your critical path is (specify each module/entity/component that this path goes through).

maximum frequency: 36.44MHz



[2.a.ii] Draw a simple schematic showing how you could implement stalling and flushing operations given an ideal N-bit register.



[2.a.iii] Create a testbench that instantiates all four of the registers in a single design. Show that values that are stored in the initial IF/ID register are available as expected four cycles later, and that new values can be inserted into the pipeline every single cycle. Most importantly, this testbench should also test that each pipeline register can be individually stalled or flushed.

tb_PipelineregisterUpdate.vhd



To start, stall nothing, so you can see how the data (0x00000004) is loaded into each register (1-4).

And this continues when a new data value (0x00000002) is then loaded into each register.

And then we try to load in a new value (0x00000003) into all the registers again, but register 2 and register 4 are stalled, so the value is loaded only into register 1, and none of the other registers change their values.

Finally, register 3 gets flushed, and you can see its value get overwritten with 0x00000000.

The pattern continues normally, with only the first register being updated because it is the only one that is not stalled and receiving new values.

[2.b.i] list which instructions produce values, and what signals (i.e., bus names) in the pipeline these correspond to.

R-TYPE

EX/MEM:

- s_ALUOut
- EX_stage_Register(63 downto 32)
- s_DMemAddr
- s_wireWBmux

MEM/WB:

- wireTOWB
- MEM_stage_Register(31 downto 0)
- s_RegWrData

I-TYPE

EX/MEM:

- s_ALUOut
- EX_stage_Register(63 downto 32)
- s_DMemAddr
- EX_stage_Register(114 downto 110)
- EX_stage_Register(115)

MEM/WB:

- s_wireWBmux
- wireTOWB
- MEM_stage_Register(31 downto 0)
- s_RegWrData
- MEM_stage_Register(50 downto 46)
- MEM_stage_Register(51)

U-type

lui

EX/MEM:

- s_ALUOut
- EX_stage_Register(63 downto 32)
- s_DMemAddr

MEM/WB:

- s_wireWBmux
- wireTOWB
- MEM_stage_Register(31 downto 0)
- s_RegWrData

auipc

EX/MEM:

- s_ALUOut
- EX_stage_Register(63 downto 32)
- s_DMemAddr

MEM/WB:

- s_wireWBmux
- wireTOWB

- MEM_stage_Register(31 downto 0)
- s_RegWrData

OTHER

lw/lb/lh/lbu/lhu

EX/MEM:
- s_ALUOut
- EX_stage_Register(63 downto 32)
- EX_stage_Register(114 downto 110)
- EX_stage_Register(115)

MEM/WB:
- s_DMemOut
- s_wireWBmux
- wireTOWB
- MEM_stage_Register(31 downto 0)
- s_RegWrData
- MEM_stage_Register(50 downto 46)
- MEM_stage_Register(51)

jalr/jal

EX/MEM:
- s_pcPLUS4
- EX_stage_Register(31 downto 0)
- EX_stage_Register(118)
- EX_stage_Register(115)

MEM/WB:
- anotherWireYay
- wireTOWB
- MEM_stage_Register(31 downto 0)
- s_RegWrData

[2.b.ii] List which of these same instructions consume values, and what signals in the pipeline these correspond to.

R-TYPE

IF/DE:
- IF_stage_Register(31 downto 0)

DE/EX:
- s_Inst_19to15
- s_Inst_24to20
- register1TOALU
- register2TOALU
- ID_stage_Register(114 downto 83)
- ID_stage_Register(146 downto 115)

I-TYPE

IF/DE:
- IF_stage_Register(31 downto 0)

DE/EX:
- s_Inst_19to15

- register1TOALU
- extenderTOMUX
- ID_stage_Register(114 downto 83)
- ID_stage_Register(178 downto 147)

U-TYPE

lui

IF/DE:

- IF_stage_Register(31 downto 0)

DE/EX:

- ID_stage_Register(178 downto 147)
- extenderTOMUX

auipc

IF/DE:

- IF_stage_Register(31 downto 0)
- IF_stage_Register(95 downto 64)

DE/EX:

- ID_stage_Register(178 downto 147)
- extenderTOMUX
- ID_stage_Register(210 downto 179)

OTHER

lw/lb/lh/lbu/lhu

IF/DE:

- IF_stage_Register(31 downto 0)

DE/EX:

- s_Inst_19to15
- register1TOALU
- extenderTOMUX
- ID_stage_Register(114 downto 83)

jalr/jal

IF/DE:

- IF_stage_Register(31 downto 0)
- IF_stage_Register(95 downto 64)

DE/EX:

- ID_stage_Register(210 downto 179)
- extenderTOMUX

[2.b.iii] generalized list of potential data dependencies. From this generalized list, select those dependencies that can be forwarded (write down the corresponding pipeline stages that will be forwarding and receiving the data), and those dependencies that will require hazard stalls.

OMIT, no forwards, so we are only stalling

[2.b.iv] global list of the datapath values and control signals that are required during each pipeline stage

IF Stage

```
 i_D(31 downto 0) => s_Inst, --Cur
 i_D(63 downto 32) => currentInst,
```

ID Stage

```
i_D(31 downto 0) => IF_stage_Register(63 downto 32),
i_D(32) => Branch_Wire, --Branch_Wire
i_D(33) => Wire_ALUSrc, --Wire_ALUSrc
i_D(37 downto 34) => Wire_ALUControl, --Wire_ALUContr
i_D(40 downto 38) => Wire_ImmType, --Wire_ImmType
i_D(41) => Wire_ResultSrc, --Wire_ResultSrc
i_D(42) => Wire_MemWrite, --Wire_MemWrite
i_D(43) => Wire_PCSrc, --Wire_PCSrc
i_D(44) => Wire_RegDst, --Wire_RegDst
i_D(45) => halt_Wire, --halt_Wire
i_D(46) => s_WritePCORResultSrc, --s_WritePCORResultS
i_D(47) => s_ALUOutORPC, --s_ALUOutORPC
i_D(48) => s_PCAddress, --s_PCAddress
i_D(49) => Wire_RegWrite, --Wire_RegWrite
i_D(50) => s_zeroMuxWire, --s_zeroMuxWire
i_D(82 downto 51) => doINeedAWireHere, --zer
i_D(114 downto 83) => register1TOALU, --Data 1
i_D(146 downto 115) => register2TOALU, --Data 2
i_D(178 downto 147) => IF_stage_Register(31 downto 0)
```

EX Stage

```
i_D(31 downto 0) => s_pcPLUS4, --s_pcPLUS4
i_D(63 downto 32) => s_ALUOut, --ALu out
i_D(95 downto 64) => ID_stage_Register(146 downto 115),
i_D(102 downto 96) => ID_stage_Register(153 downto 147),
i_D(109 downto 103) => ID_stage_Register(178 downto 172)
i_D(114 downto 110) => ID_stage_Register(158 downto 154)
i_D(115) => ID_stage_Register(49), --Reg Write
i_D(116) => ID_stage_Register(42), --Mem Write
i_D(117) => ID_stage_Register(41), --ResultSRC
i_D(118) => ID_stage_Register(46), --WritePCORResultSRC
```

MEM Stage

```
i_D(31 downto 0) => wireTOWB, --Final data value for r
i_D(38 downto 32) => EX_stage_Register(102 downto 96),
i_D(45 downto 39) => EX_stage_Register(109 downto 103)
i_D(50 downto 46) => EX_stage_Register(114 downto 110)
i_D(51) => EX_stage_Register(115), --write enable sign
```

[2.c.i] list all instructions that may result in a non-sequential PC update and in which pipeline stage that update occurs.

BRANCH INSTRUCTIONS

- The branch control signal is set to high during the Execute stage, and in the follow stage the PC address updates.

JAL

- The PCSrc control signal is set to high during the Execute stage, and in the follow stage the PC address is updated according to the jump.

JALR

- The PCSrc control signal is set to high during the Execute stage, and in the follow stage, the PC address updates with the ALUOut value.

[2.c.ii] For these instructions, list which stages need to be stalled and which stages need to be squashed/flushed relative to the stage each of these instructions is in.

BRANCH INSTRUCTIONS

- For a branch at any point, flush the ID and EX register in the following clock cycle.

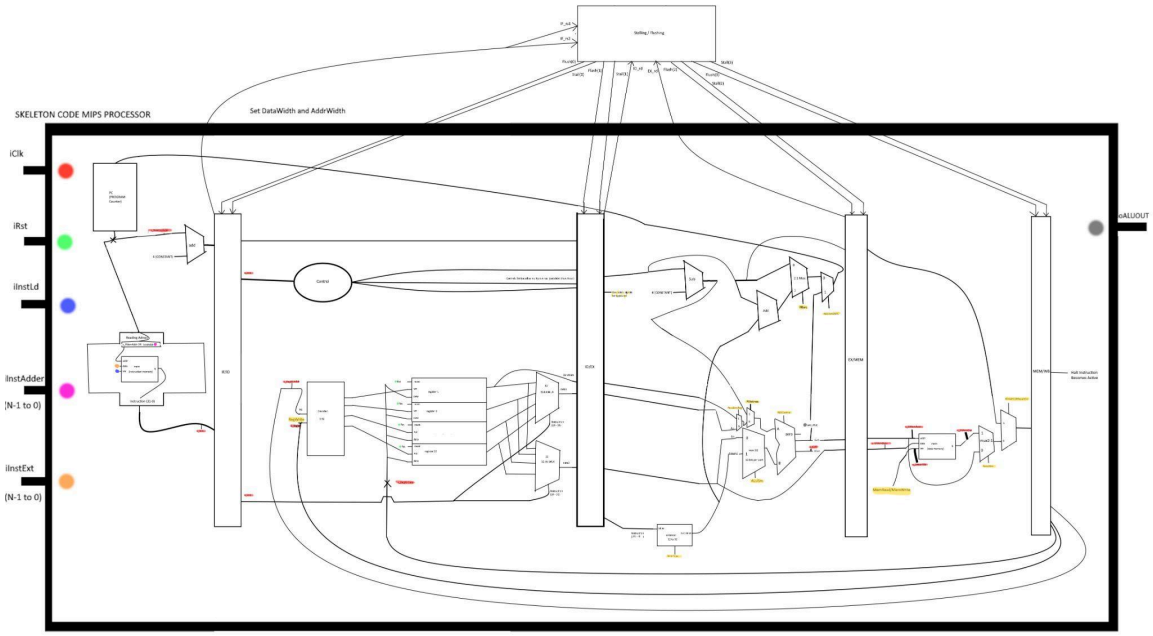
JAL

- For a jal, we flush the IF and ID registers on the second cycle and stall the IF register with that bubble value.

JALR

- For jalr, we flush the second register and then stall the first register in the following clock cycle.

[2.d] implement the hardware-scheduled pipeline using only structural VHDL. As with the previous processors that you have implemented, start with a high-level schematic drawing of the interconnection between components.



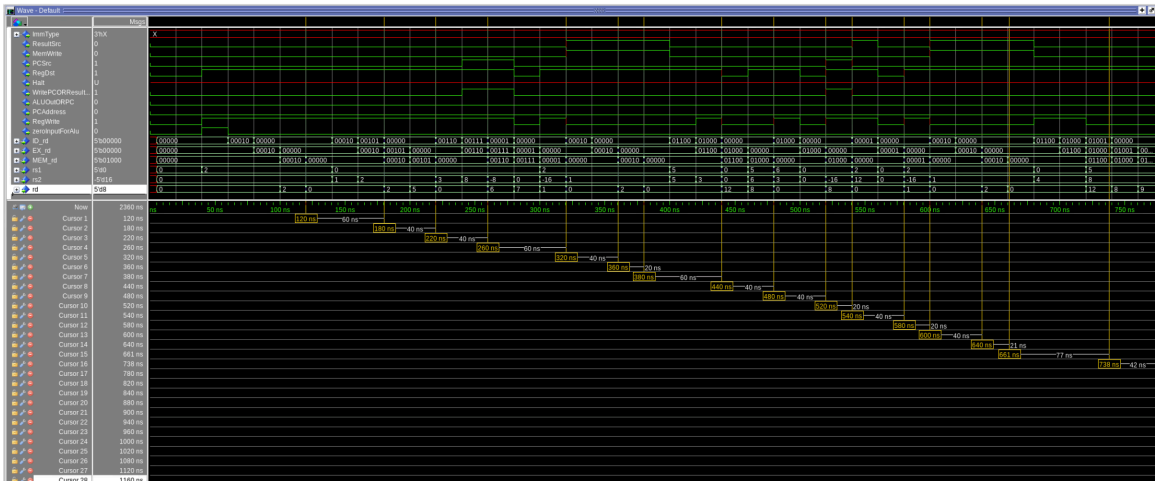
[2.e – i, ii, and iii] In your writeup, show the QuestaSim output for each of the following tests, and provide a discussion of result correctness. It may be helpful to also annotate the waveforms directly.

[2.e.i] Create a spreadsheet to track these cases and justify the coverage of your testing approach. Include this spreadsheet in your report as a table.

[2.e.ii] Create a spreadsheet to track these cases and justify the coverage of your testing approach. Include this spreadsheet in your report as a table.

- **OMIT Section E and include waveforms for base test and control flow test**

1. Base Test

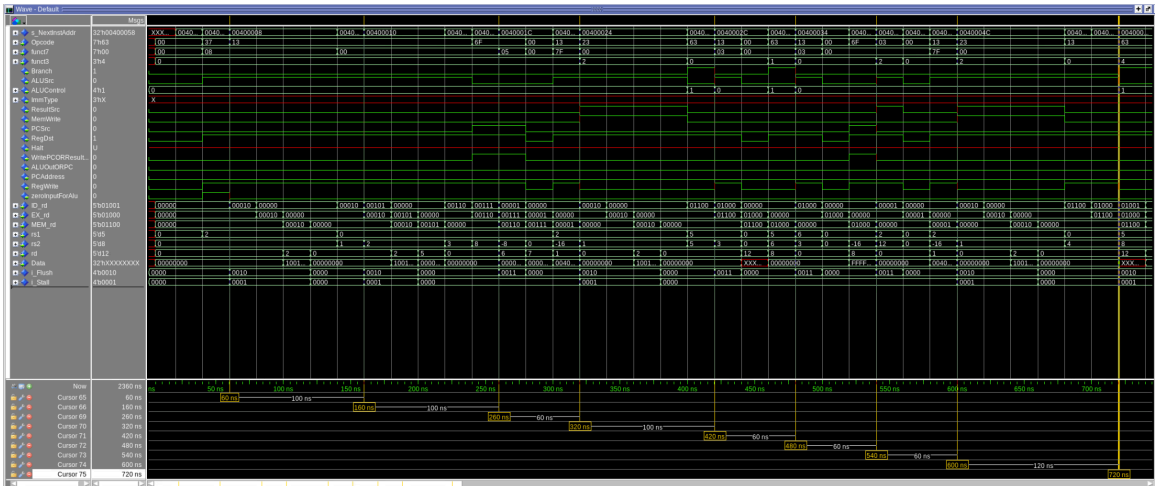


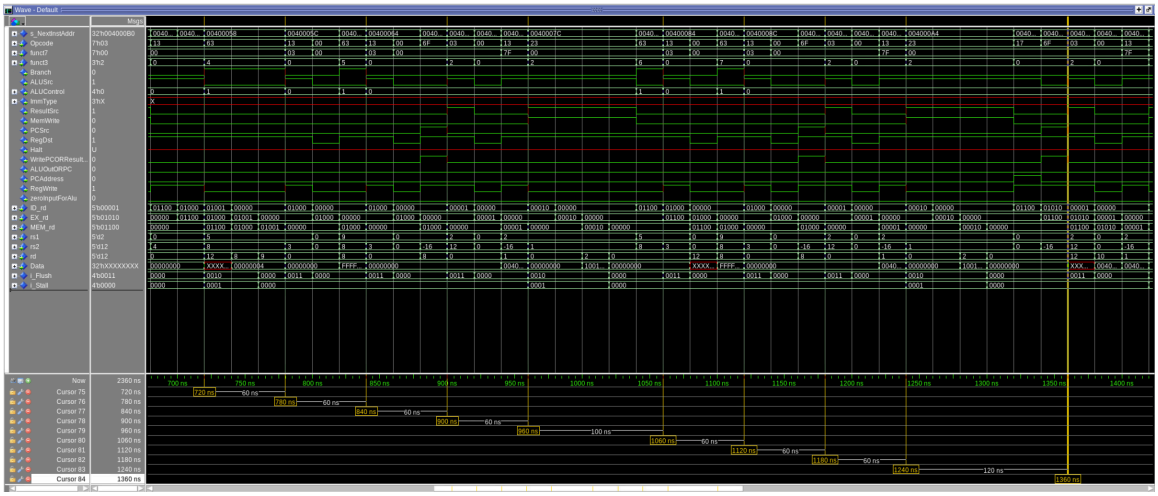
Given the waveform above, you can see how, as the NextInstAddress increases, the opcode changes for each new instruction, and the corresponding rd, and ALU control signals change. It will take approximately 5 cycles before we see the results from our instructions being loaded into our pipeline. After that, we will be able to examine the values to ensure that we are getting the correct output, as we did with our previous waveforms.

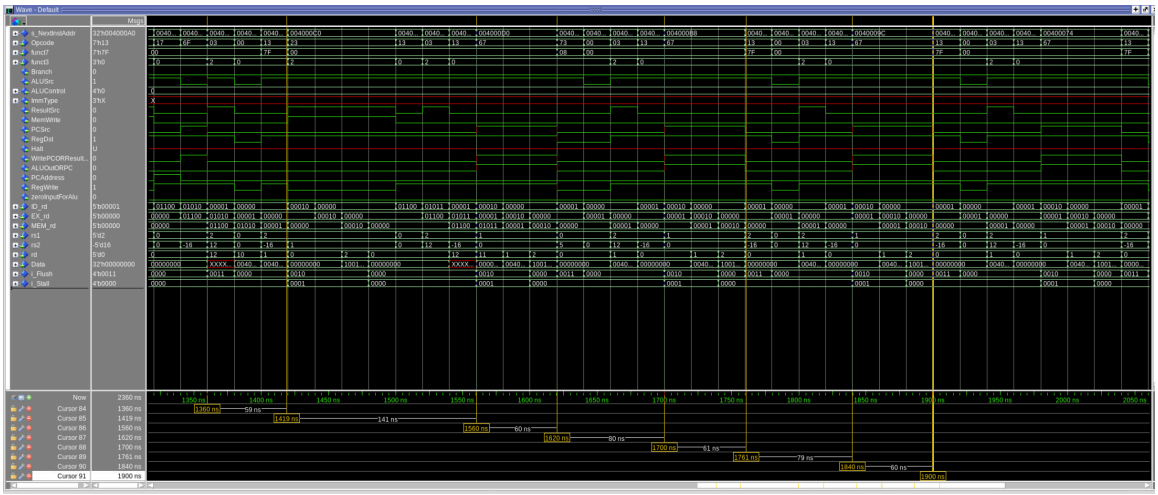
The only sections where the instructions are not executed as they typically would be on a RISC-V single-cycle processor are during our stalls and flushes. You can see when the i_stall and i_flush instructions are set to high from the signals (each digit corresponds to the register from left to right; for example, Register 1 is bit 0, Register 2 is bit 1, and so on). When those signals are set too high, the results will be seen in the next cycle. I marked those locations with cursors to make them easier to see. So, a cursor marks the start of either a flush or a stall.

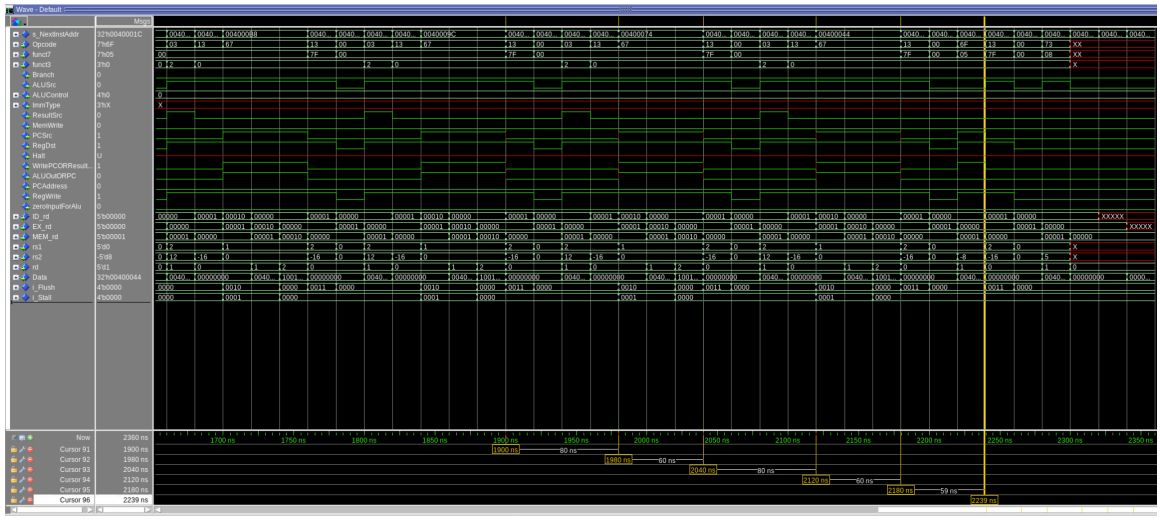
When these stalls occur, we double-check that on the next cycle, the value for the corresponding register either got properly flushed or properly stalled. You should also be able to see the register from both in front and behind, and react accordingly.

For example, you can see how our RD value for our first instruction writes then the stalls from the cursors before it. This behavior is consistent across all of our waveform cursors until the WFI instruction is finally read and the halt instruction is set to high.


Wave - Default
s_NextInstrAddr
Opcode
funct7
funct3
Branch
ALUSrc
ALUControl
ImmType
ResultSrc
MemWrite
PCSrc
RegDst
Halt
WritePCORResult...
ALUOutORPC
PCAddress
RegWrite
zeroInputForAlu
ID_rd
EX_rd
MEM_rd
rs1
rs2
rd
Data
i_Flush
i_Stall
Now
Cursor 65
Cursor 66
Cursor 69
Cursor 70
Cursor 71
Cursor 72
Cursor 73
Cursor 74
Cursor 75


Wave - Default
s_NextInstrAddr
Opcode
funct7
funct3
Branch
ALUSrc
ALUControl
ImmType
ResultSrc
MemWrite
PCSrc
RegDst
Halt
WritePCORResult...
ALUOutORPC
PCAddress
RegWrite
zeroInputForAlu
ID_rd
EX_rd
MEM_rd
rs1
rs2
rd
Data
i_Flush
i_Stall
Now
Cursor 75
Cursor 76
Cursor 77
Cursor 78
Cursor 79
Cursor 80
Cursor 81
Cursor 82
Cursor 83
Cursor 84


Wave - Default
s_NextInstrAddr
Opcode
funct7
funct3
Branch
ALUSrc
ALUControl
ImmType
ResultSrc
MemWrite
PCSrc
RegDst
Halt
WritePCORResult...
ALUOutORPC
PCAddress
RegWrite
zeroInputForAlu
ID_rd
EX_rd
MEM_rd
rs1
rs2
rd
Data
i_Flush
i_Stall
Now
Cursor 84
Cursor 85
Cursor 86
Cursor 87
Cursor 88
Cursor 89
Cursor 90
Cursor 91



2. Control Flow Test

The purpose of this test was to confirm that my processor would still execute the correct instructions with stalls and flushes, even when presented with a long series of instructions that included jumps and branches.

This is the same situation as before, where we know an instruction is correct if, after a couple of cycles, the opcode, RD, and data being written match. Each cursor represents a stall or flush that occurs during program execution. The stall/flush, just like last time, goes into effect after the next rising edge.

Although it is not the main purpose of this waveform, because we jump around so much, it can be hard to know which instruction is being executed correctly when. To help mitigate this, a list is provided below that indicates the instruction being executed immediately before each JAL and branch, as well as the instruction immediately after each JALR.

The instruction right before each JAL and branch

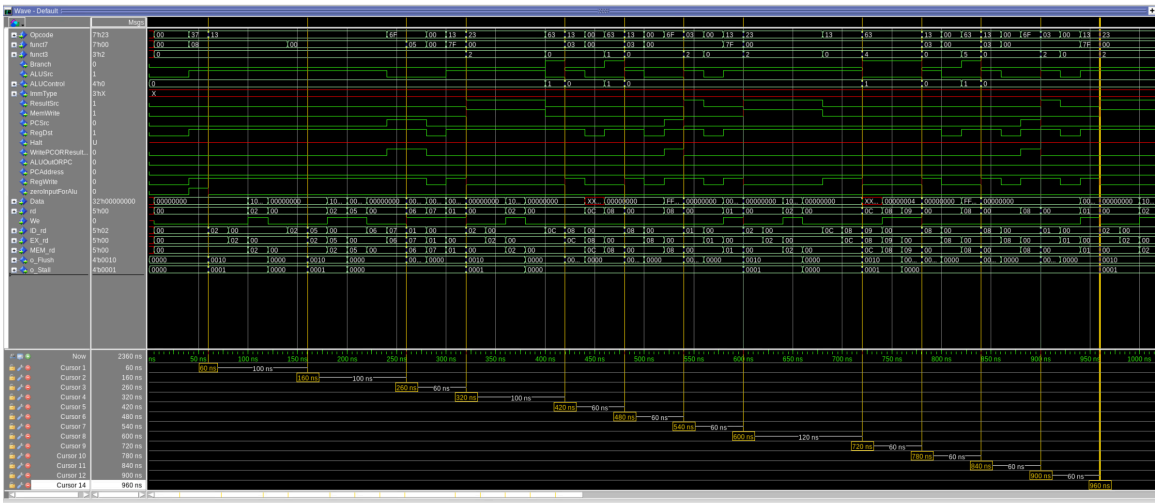
Addi x7, x0, 3
Sw x1 12(x2)
Beq x5, x5, func1_beq
Bne x5, x6, func1_bne
Addi x9, x0, 4
Blt x5, x8, func2_blt
Bfe x9, x8, func2_bge
Sw x1, 12 (x2)
Bltu x5, x8, fun3_bltu
Bgeu x9, x8, func3_bgeu
Auipc x10, 0

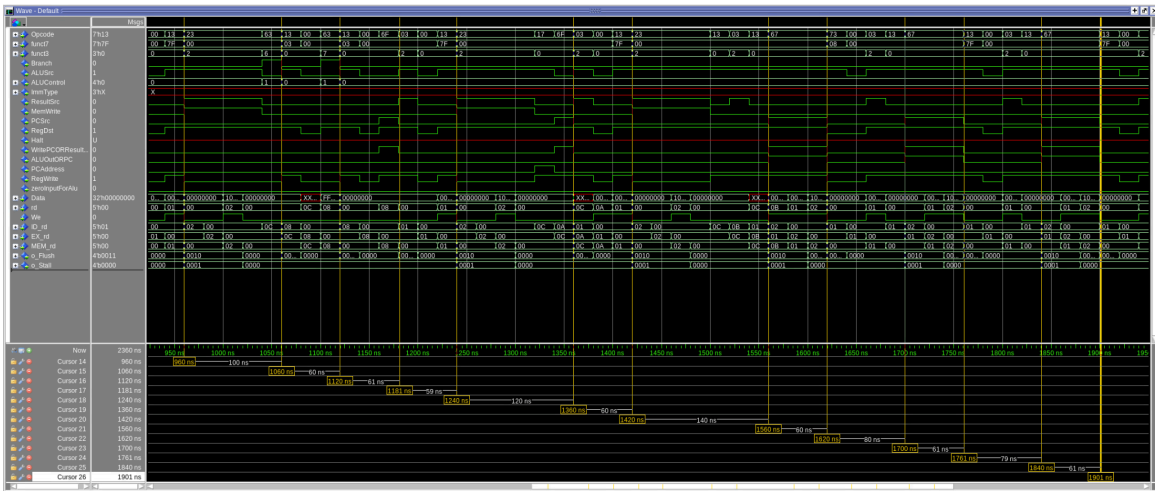
After the Auipc instruction, only the jalr control flow instructions should be executing up until our final jal.

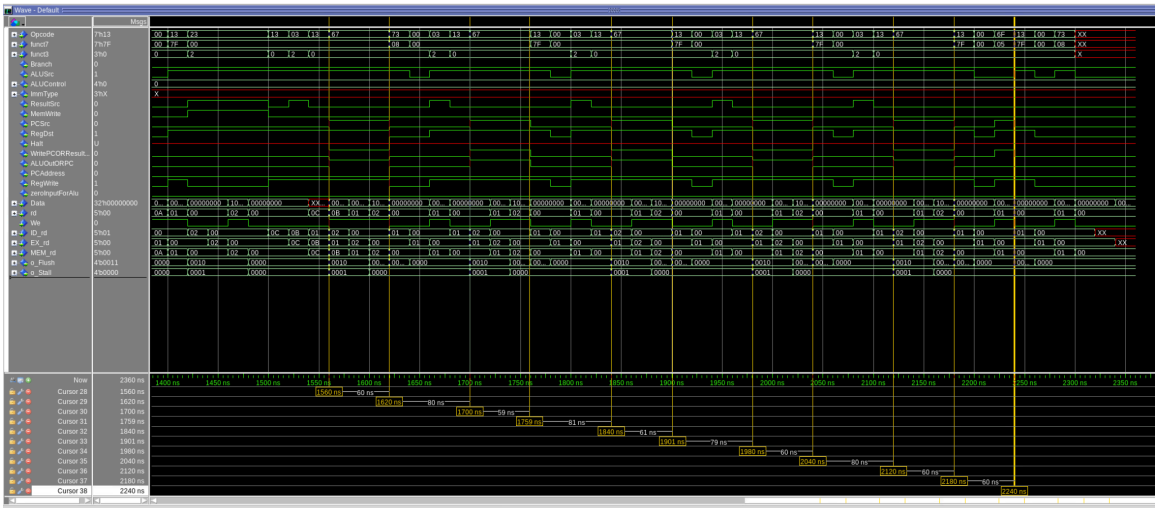
The instruction right after each jalr

Lw x1, 12(x2)
Lw x1, 12(x2)
Lw x1, 12(x2)
Lw x1, 12(x2)
Jal x1, end

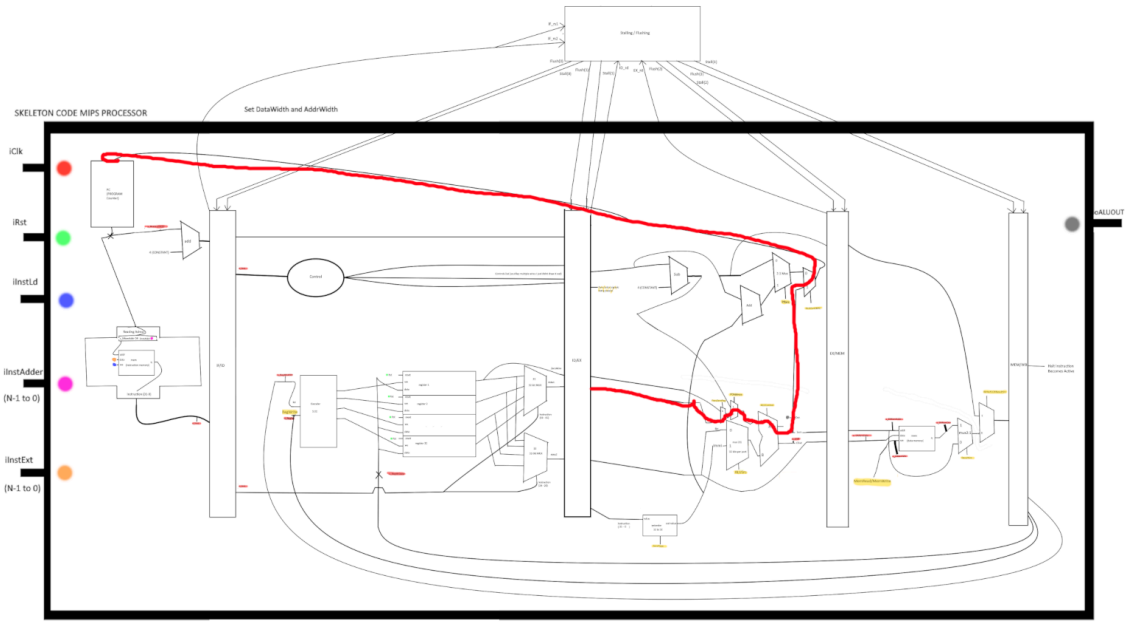
All instructions after the jalr should match the instructions above, and after the final jal, we should encounter a WFI instruction, after which our program ends, as halt is set to 1.







[2.f] report the maximum frequency your hardware-scheduled pipelined processor can run at and determine what your critical path is (specify each module/entity/component that this path goes through).



Maximum Frequency: 44.05 MHz